Machine Learning Assignment 2

End-to-End Multiclass Classification of Dry Bean Varieties

Student Name	<ADD STUDENT NAME>
Student ID	<ADD BITS ID>
Programme	M.Tech Artificial Intelligence and Machine Learning
Course	Machine Learning
GitHub Repository	https://github.com/aakashK0112/ML_Assignment_2
Streamlit Application	<ADD STREAMLIT DEPLOYMENT URL>

Prepared as an original, reproducible academic submission. Experimental values in this report were calculated from the bundled data and saved models.

1. Problem Statement

The objective is to classify dry bean grains into seven registered varieties using 16 geometric and shape measurements. Five standard classifiers are trained on the same training split, compared with six metrics, persisted with their preprocessing, and exposed through a Streamlit evaluation dashboard.

2. Dataset Description and Source

The UCI Dry Bean dataset contains 13,611 observations, 16 numeric input features, and the target Class. It represents BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER, and SIRA. The raw file contains 0 missing values and 68 exact duplicates. After duplicate removal, 13,543 rows remain.

Source: UCI Machine Learning Repository, Dry Bean dataset. DOI: 10.24432/C50S4B. License: CC BY 4.0. URL: https://archive.ics.uci.edu/dataset/602/dry+bean

3. Preprocessing and Experimental Design

Exact duplicates are removed before splitting. A stratified 80:20 split with random_state=42 produces 10,834 training records and 2,709 held-out test records. Median imputation is fit only on training data. StandardScaler is included for Logistic Regression, KNN, Gaussian Naive Bayes, and SVM; the Decision Tree is not scaled. Every transformer and estimator is stored in one scikit-learn Pipeline, preventing test-data leakage.

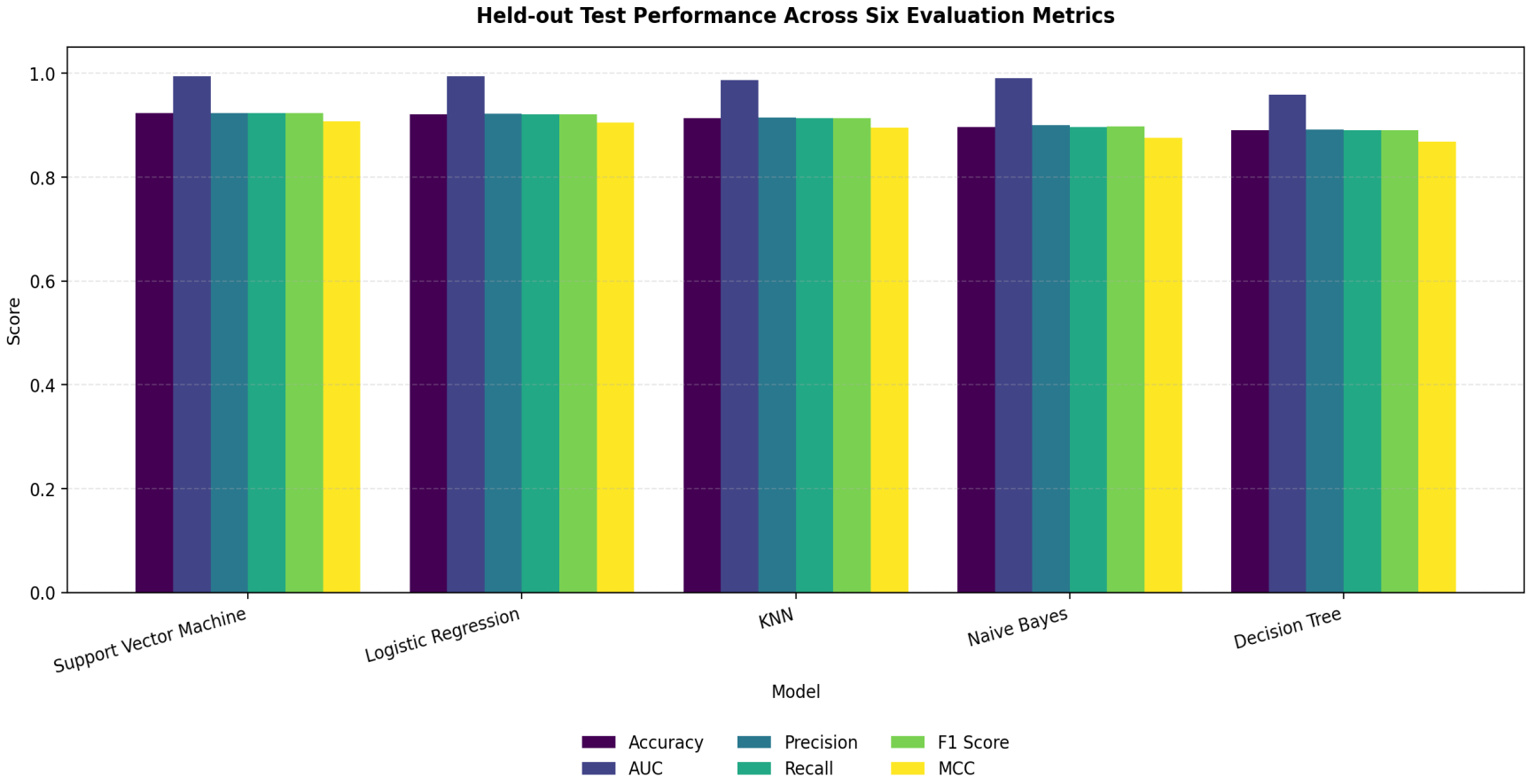
4. Model Implementation Summary

Logistic Regression provides a regularized linear baseline. The Decision Tree captures nonlinear rules with depth and leaf constraints. KNN uses distance weighting after standardization. Gaussian Naive Bayes models per-class feature distributions. The RBF SVM uses nonlinear margins with probability=True for multiclass AUC.

5. Model Comparison

All scores below were calculated on the same held-out test set. AUC is macro one-vs-rest; Precision, Recall, and F1 are weighted averages.

Model	Accuracy	AUC	Precision	Recall	F1	MCC	Mean Rank
Support Vector Machine	0.9236	0.9946	0.9237	0.9236	0.9236	0.9075	1.1667
Logistic Regression	0.9210	0.9948	0.9216	0.9210	0.9211	0.9045	1.8333
KNN	0.9133	0.9872	0.9142	0.9133	0.9134	0.8951	3.1667
Naive Bayes	0.8970	0.9911	0.8997	0.8970	0.8972	0.8762	3.8333
Decision Tree	0.8907	0.9589	0.8915	0.8907	0.8910	0.8680	5.0000


Held-out Test Performance Across Six Evaluation Metrics
Score
1.0
0.8
0.6
0.4
0.2
0.0
Support Vector Machine
Logistic Regression
KNN
Naive Bayes
Decision Tree
Model
Accuracy
AUC
Precision
Recall
F1 Score
MCC

6. Model-wise Observations

Logistic Regression

Strong linear baseline with 0.9210 accuracy and 0.9948 macro AUC. Its high AUC shows excellent class separation, while its F1 and MCC remain close to the best model.

Decision Tree

The most interpretable nonlinear model, but the lowest balanced result: 0.8910 weighted F1 and 0.8680 MCC. The gap suggests higher variance than the scaled models.

KNN

Distance-weighted KNN reached 0.9133 accuracy and 0.8951 MCC after standardization, giving a solid nonlinear result without an explicit parametric boundary.

Naive Bayes

Gaussian Naive Bayes achieved 0.9911 macro AUC, but its 0.8972 F1 score trails the top models. Correlated geometric features make its conditional-independence assumption restrictive.

Support Vector Machine

Best balanced performer with 0.9236 accuracy, 0.9236 F1, and 0.9075 MCC. The RBF kernel captures nonlinear class boundaries while retaining excellent probability-based AUC.

7. Overall Winning Model

Support Vector Machine is selected using a balanced rank across Accuracy, AUC, weighted Precision, weighted Recall, weighted F1, and MCC rather than accuracy alone. Its mean rank is 1.1667. The result reflects consistent discrimination, class-balanced predictive quality, and strong agreement beyond chance.

8. Streamlit Application Features

The application loads all five saved joblib pipelines and never retrains them. It validates an uploaded labelled CSV, previews the data, reports six metrics, renders a confusion matrix and classification report, summarizes actual versus predicted classes, compares all five models on the uploaded test set, and offers downloadable predictions. Invalid or incomplete files receive a clear error message.

9. Reproducibility and Quality Controls

The included scripts recreate the fixed split, train every model, save all outputs, and regenerate this report. Model files contain preprocessing and estimator objects together. Source files use project-relative paths, making the folder portable across Windows and Streamlit Community Cloud. The held-out test CSV contains all 16 input features plus Class.

10. GitHub README Content Summary

The repository README documents the problem, dataset and source, preprocessing, five models, metric definitions, actual comparison table, model observations, winning-model rationale, Streamlit features, installation, local execution, cloud deployment, folder structure, conclusion, and placeholders that must be replaced after deployment.

Repository: https://github.com/aakashK0112/ML_Assignment_2

Deployment placeholder: <ADD STREAMLIT DEPLOYMENT URL>

11. BITS Virtual Lab Execution Evidence

Run RUN_ON_BITS_MACHINE.bat or follow the README commands on the BITS machine. After Streamlit opens, capture the terminal showing the local URL and the dashboard showing the selected model metrics.

INSERT BITS VIRTUAL LAB EXECUTION SCREENSHOT HERE

Suggested evidence: terminal command, successful Streamlit URL, dashboard metric cards, and model comparison view.

Before final submission, replace all student, GitHub, Streamlit, and screenshot placeholders. No URL or execution evidence has been fabricated.

12. Conclusion

The assignment delivers a reproducible multiclass classification workflow over a real UCI dataset. The RBF SVM is the strongest balanced model, with Logistic Regression a close and simpler alternative. The saved-pipeline architecture ensures that every local, uploaded, or cloud prediction receives exactly the transformations learned from training data.

13. Key Files

app.py - Streamlit interface; train_models.py - reproducible training and evaluation; generate_report.py - README and PDF generation; dataset.csv - full UCI data; test_data.csv - held-out data; models/ - five saved pipelines; results/ - comparison, reports, audit, and confusion matrices; assets/ - performance chart.

14. Submission Checklist

[ ] Add student name and BITS ID. [ ] Create GitHub repository and replace its URL. [ ] Deploy Streamlit and replace its URL. [ ] Run on BITS Virtual Lab and insert an authentic screenshot. [ ] Recheck the generated PDF and repository contents before upload.